

Refactoring Plan: LTC / SRNN / CT-RNN → Modern GPU-Accelerated Framework

Last updated: 2026-03-24 — decisions finalized

1. Problem Statement

The current [liquid_time_constant_networks](#) codebase is built on **TensorFlow 1 running in TF2 v1-compatible mode**, uses tiny UCI datasets, trains on CPU-only VMs, and has models hardcoded at 32 neurons. Separately, the [Intersect-LNNs-SRNNs](#) project has a Matlab reservoir-computing implementation and a Julia port with 27 individual training scripts. Both codebases share the same structural problems: massive code duplication across experiments, no GPU acceleration, and no path to scale.

We need a **v3 codebase** that: - Runs on **NVIDIA L4 GPUs** (SM89, 24 GB GDDR6) on GCP Compute Engine, with A100/H100 as stretch targets - Supports **batched model parallelism** (`torch.bmm` / `jax.vmap`) for ablation sweeps - Trains on **challenging, community-standard datasets** (not just UCI toys) - Can interoperate with or benchmark against **Mamba-2/3, Gated Delta Networks, and S4/S5** - Has a clean modular architecture: models, datasets, training loop, and cloud deployment are separated

2. What Currently Exists

2.1 Legacy Python / TF1 Codebase

Item	Details
Location	liquid_time_constant_networks/experiments_with_ltc
Framework	<code>tensorflow.compat.v1</code> with <code>tf.disable_v2_behavior()</code>
Models	ltc_model.py (LTC with 3 ODE solvers), ctrnn_model.py (CT-RNN, NODE, CT-GRU), srnn_model.py (SRNN with SFA/STD, Dale's law, sparsity masks)
Model variants	15+ ablations (srnn, srnn-per-neuron, srnn-echo, srnn-no-adapt, srnn-no-adapt-no-dales, srnn-sfa-only, srnn-std-only, srnn-E-only, srnn-e-only-echo, srnn-e-only-per-neuron, hopf, lstm, ltc, ltc_rk, ltc_ex, node, ctgru, ctrnn)
Model size	Hardcoded at 32 neurons
Experiments	10 separate Python scripts, one per dataset, each copy-pasting the full model-selection switch and training loop — see e.g. har.py , cheetah.py
Cloud infra	CPU-only <code>n2-standard-2</code> spot VMs via launch_all_fast.sh ; GCS results bucket; self-deleting VMs
GPU usage	None — <code>os.environ['CUDA_VISIBLE_DEVICES'] = '-1'</code> in every script

2.2 Julia / Matlab Codebase

Item	Details
Location	Intersect-LNNs-SRNNs/JuliaLang
Models	srnn.jl (29 KB, full SRNN), ltc1.jl / ltc2.jl (LTC variants)
Training	27 individual scripts (one per dataset × model), also duplicated
Matlab	CLAUDE.md : SRNN + LNN reservoir models with ESN memory capacity measurement

2.3 Datasets

Current datasets are all **small, short-sequence UCI benchmarks** from [download_datasets.sh](#):

Dataset	Type	Input dim	Sequence length	Size
HAR	6-class clf	561	16	10K samples
Gesture	5-class clf	18	16	10K
Occupancy	2-class clf	5	16	20K
sMNIST	10-class clf	1	784	70K images
Traffic	regression	7	16	48K
Power	regression	7	32	2M (short seqs)
Ozone	2-class clf	72	16	2.5K
Person	5-class clf	4	32	164K
Cheetah	17-dim regression	17	32	25× .npz files

Except for sMNIST (784 steps), all sequences are **16–32 steps** — far too short to stress long-range dependency modeling.

3. Target Hardware: NVIDIA L4 on GCP

Primary target: **g2-standard-16** (1× L4 GPU, 16 vCPUs, 64 GB RAM) on GCP Compute Engine.

Spec	L4	A100 (40GB PCIe)	H100 (PCIe)
Architecture	Ada Lovelace	Ampere	Hopper
Compute Capability	SM89	SM80	SM90
VRAM	24 GB GDDR6	40 GB HBM2e	80 GB HBM3
Memory BW	~300 GB/s	~2,039 GB/s	~3,350 GB/s
TDP	72W	250W	350W
GCP cost (on-demand)	~\$0.70/hr	~\$2.90/hr	~\$8–10/hr
GCP availability	Easy	Quota required	Rare

L4 is ideal for our workload because: - 24 GB is sufficient for models up to ~256–512 neurons with full ablation batching - SM89 has full BF16/FP16 tensor core support - Cost-effective for long training runs (~\$0.70/hr vs \$2.90+ for A100) - High GCP availability without special quotas - Forward-compatible: code that runs on L4 (SM89) will run on A100 (SM80) and H100 (SM90)

For MAMBA-3 specifically, L4 supports optimized prefill kernels (Triton + TileLang) but may need the `mamba3-minimal` PyTorch fallback for decode kernels — see [mamba3-gpu-compatibility-t4-l4-a100.md](#).

4. Framework Decision: PyTorch vs JAX vs Julia

4.1 Comparison Matrix

Criterion	PyTorch	JAX + Diffrax	Julia SciML
Mamba-2/3 support	Native (<code>mamba-ssm</code>)	Community ports	None
Gated Delta Networks	Official impl	Minimal	None
Neural ODE solvers	<code>torchdiffeq</code> , TorchDyn	Diffrax (best quality)	DiffEqFlux (most diverse)
bmm / batched ops	<code>torch.bmm</code> , <code>vmap</code>	<code>jax.vmap</code> (better JIT)	Manual batching
GPU ecosystem	Mature CUDA	XLA/CUDA/TPU	<code>CUDA.jl</code> works but less mature
L4 compatibility	SM89 fully supported	SM89 fully supported	Supported via <code>CUDA.jl</code>
Community/packages	Largest	Growing fast	Small ML community
Debugging	Eager by default	JIT tracing is opaque	Interactive
Pre-built LTC/CfC	<code>ncps</code> package	None	None
Stiff ODE solvers	Limited	Diffrax has some	300+ solvers

4.2 Decision: PyTorch + torchdiffeq (primary)

DECIDED: PyTorch is the primary framework. Julia is set aside for now (may revisit for adjoint method experiments later). Everything uses discretized methods (Euler, RK4, semi-implicit).

Rationale: 1. **Mamba-2/3 and Gated Delta Networks are PyTorch-native.** These are the primary SSM baselines we need to compare against. Running them in JAX or Julia would require reimplementing. 2. **ncps** (Neural Circuit Policies, Hasani et al.) provides production-quality LTC and CfC cells in PyTorch out of the box. 3. **torch.vmap + torch.bmm** enables batched ablation sweeps — running multiple model instances (different seeds, hyperparameters, ablation configs) as a single batched operation on one GPU. 4. **PyTorch on L4** is the most thoroughly tested and documented path. GCP’s Deep Learning VMs ship with PyTorch pre-installed. 5. Our CT-RNN / SRNN models use **discretized dynamics** (explicit Euler or semi-implicit), not adaptive ODE solvers — so Julia’s stiff solver advantage is less relevant. The RNN step is a fixed-step integration, which is straightforward in PyTorch.

Configuration system: Hydra. Hydra provides hierarchical YAML configs with command-line overrides, multi-run sweeps (`--multirun`), and structured experiment composition. This is a natural fit for managing model variants, dataset configs, and ablation sweeps.

Julia is set aside for now. The existing Julia/Matlab codebases in [Intersect-LNNs-SRNNs](#) remain available for reservoir computing research and potential adjoint method experiments later. - See [Julia_ecosystem_analysis_opus.md](#) for detailed Julia analysis - See [MAMBA3_tech_stack.md](#) for MAMBA3-specific stack

5. Datasets: From UCI Toys to Community Benchmarks

5.1 Design Principles

DECIDED: Three tiers of datasets — smoke tests (HAR, Cheetah), standard benchmarks (ETTh1, Electricity, AMASS, Speech Commands, sCIFAR, sMNIST), and a domain-specific electrophysiology dataset.

We need **three tiers** of datasets: - **Tier 0 (Smoke tests):** Small, fast, for CI/pipeline validation. HAR (classification) and Cheetah (vector autoregression) — both are already implemented and train quickly. - **Tier 1 (Standard benchmarks):** Challenging, long-sequence, multivariate datasets that are standard in the SSM/recurrent model literature (Mamba, S4, Gated Delta Networks). - **Tier 2 (Domain-specific):** Electrophysiology data that uniquely plays to our models’ continuous-time / biologically-inspired inductive biases.

5.2 Datasets

Tier 0: Smoke Tests (keep from legacy)

Dataset	Task	Why keep
HAR	6-class activity recognition	Quick classification pipeline sanity check. Short sequences, fast epochs.
Cheetah	17-dim vector autoregression	Quick regression/autoregression pipeline sanity check. Already implemented, trains fast.

Tier 1: Standard Benchmarks

Dataset	Task	Input dim	Sequence length	Size	Why this dataset
sMNIST	10-class sequential classification	1	784	70K images	The most widely-used sequence classification benchmark. 784 steps. Universal comparability. A final benchmark dataset (not a smoke test).

Dataset	Task	Input dim	Sequence length	Size	Why this dataset
ETTh1	Multivariate 7 time series forecast-ing		336→720	17,420 hourly steps	Standard LTSF benchmark. Used by Mamba variants (TimeMamba, TSMamba, C-Mamba), Informer, PatchTST, and every LTSF paper since 2021. Electricity transformer oil temperature + 6 load features. Forecasting horizons of 96/192/336/720 steps.
Electricity (ECL)	Multivariate 321 forecast-ing		336→720	26,304 hourly steps	High-dimensional LTSF benchmark. 321 simultaneous electricity consumption channels. Tests scalability to wide multivariate inputs. Standard in Autoformer, FEDformer, DLinear, iTransformer comparisons.
AMASS	Motion capture autore-gression	159 (SMPL pose params)	200–2000+	40+ hours, 344 subjects	Multivariate autoregression benchmark. Unifies 15 MoCap datasets into SMPL body model parameters. 3D human body joint prediction with complex nonlinear dynamics. Natural successor to Cheetah (17-dim) but with richer dynamics (full human body, 159 pose dimensions). Account created at amass.is.tue.mpg.de . Human3.6M may be added later with academic license.
Speech Commands v2	35-class spoken word clas-sification	1 (raw audio)	16,000	105K 1-second clips	Long sequence classification. Raw audio at 16kHz = 16,000 steps. Standard S4/Mamba/LRA benchmark for testing long-range dependency capture.
sequential CIFAR-10	10-class image classifica-tion	1 (pixel-by-pixel)	1,024 (32×32) or 3,072 (3×32×32)	60K images	Synthetic long-range benchmark. Process pixels sequentially. Very common in S4, Mamba, and linear recurrence papers.

Tier 2: Electrophysiology (domain-specific)

Dataset	Task	Input dim	Sequence length	Size	Format
Intracranial EEG	Multivariate vector autoregres-sion	Up to 200 channels	512 Hz × hours	24+ hours per subject, multiple subjects	HDF5 (v7.3 Matlab format)

This is a uniquely compelling benchmark for our biologically-inspired models: - **200-channel, 512 Hz** data creates long, high-dimensional multivariate sequences - **Continuous-time** inductive biases of SRNN/LTC networks directly match the underlying neural dynamics - **Multiple subjects** enable both single-subject and cross-subject generalization experiments - For initial experiments, we can subsample (e.g., 64 channels, 1-hour segments) to keep training tractable - Full 24-hour, 200-channel runs will stress-test model scaling and GPU memory management

Why these datasets?

1. **ETTh1 + Electricity** are the *de facto* multivariate forecasting benchmarks for 2024–2026 SSM papers. Every Mamba variant, SSM, and transformer comparison uses them. Results are directly comparable to published numbers.
2. **AMASS** is the natural successor to our Cheetah dataset: same multivariate autoregression task, but much richer dynamics (a full human body in SMPL representation with 159 pose parameters vs. 17-dim simulated cheetah), longer sequences, and widely used in motion prediction. Distributed as SMPL body model parameters with 3 rotational DoF per joint.
3. **Speech Commands** and **sCIFAR** are the standard *long-sequence classification* benchmarks where S4, Mamba, and Gated Delta Networks are evaluated. They test genuine long-range dependency capture.
4. **Electrophysiology** is domain-specific and plays to our models’ unique strengths — no other benchmark suite tests continuous-time biologically-inspired models on actual neural data.

5.3 Dropped Datasets

The following legacy datasets can be retired from the primary benchmark suite:

Dataset	Reason to drop
Gesture	Very small, 18-dim, 16-step. Not used in modern literature.
Occupancy	5-dim, binary classification, trivially easy.
Traffic	7-dim, 16-step regression. Too small to be interesting.
Power	7-dim. Can be replaced by Electricity (321-dim).
Ozone	72-dim but only 2.5K samples. Severe class imbalance.
Person	4-dim, short sequences. Not a standard benchmark.

6. Architecture of the v3 Codebase

6.1 Project Structure

DECIDED: Fresh repo v3-liquid-networks/ with Hydra config system.

```
v3-liquid-networks/
|-- pyproject.toml          # uv/pip dependencies
|-- configs/               # Hydra configs
|   |-- model/
|   |   |-- srnn.yaml
|   |   |-- ltc.yaml
|   |   |-- ctrnn.yaml
|   |   |-- lstm.yaml
|   |   |-- mamba2.yaml
|   |   +-- mamba3.yaml
|   |-- dataset/
|   |   |-- har.yaml        # Tier 0 smoke test
|   |   |-- cheetah.yaml    # Tier 0 smoke test
|   |   |-- smnist.yaml     # Tier 1
|   |   |-- etth1.yaml
|   |   |-- electricity.yaml
|   |   |-- amass.yaml
|   |   |-- speech_commands.yaml
|   |   |-- scifar10.yaml
|   |   +-- ephys.yaml      # Tier 2 electrophysiology
|   +-- experiment/
|       |-- ablation_sfa.yaml
```

```

|         |-- smoke_test.yaml
|         +--- full_comparison.yaml
|-- src/
|   |-- models/
|   |   |-- base_rnn.py           # Abstract base: state_size, forward, readout
|   |   |-- srnn_cell.py         # SRNN with SFA/STD/Dale's/sparsity
|   |   |-- ltc_cell.py          # LTC cell (port from ncps or custom)
|   |   |-- ctrnn_cell.py        # CT-RNN, NODE, CT-GRU
|   |   |-- lstm_baseline.py     # Standard LSTM wrapper
|   |   |-- mamba_wrapper.py     # Mamba-2/3 via mamba-ssm
|   |   +--- model_registry.py   # Factory: name -> class
|   |-- datasets/
|   |   |-- base_dataset.py      # Abstract: load, split, iterate
|   |   |-- har.py
|   |   |-- cheetah.py
|   |   |-- smnist.py
|   |   |-- etth.py
|   |   |-- electricity.py
|   |   |-- amass.py
|   |   |-- speech_commands.py
|   |   |-- scifar10.py
|   |   +--- ephys.py           # HDF5 v7.3 Matlab electrophysiology
|   |-- training/
|   |   |-- trainer.py           # Unified training loop
|   |   |-- lr_schedule.py       # Warmup + cosine decay
|   |   |-- metrics.py           # Accuracy, MSE, MAE, F1
|   |   +--- checkpointing.py
|   |-- batching/
|   |   |-- vmap_runner.py       # Batched ablation via vmap/bmm
|   |   +--- seed_sweep.py       # Multi-seed execution
|   +--- utils/
|       |-- connectivity.py      # RMT matrix generation
|       +--- activations.py      # Piecewise sigmoid, etc.
|-- cloud/
|   |-- config.env
|   |-- launch.sh               # GPU VM launcher (g2-standard-16)
|   |-- startup.sh
|   +--- collect_results.py
|-- scripts/
|   |-- train.py                # Single entry point: python train.py model=srnn dataset=etth1
|   +--- ablation_sweep.py      # Batched ablation runner
+--- tests/
    |-- test_srnn_cell.py
    |-- test_ltc_cell.py
    +--- test_datasets.py

```

6.2 Key Design Decisions

Unified model interface All RNN cells implement:

```

class BaseRNNCell(nn.Module):
    def __init__(self, config: ModelConfig): ...
    @property
    def state_size(self) -> int: ...
    def forward(self, x: Tensor, state: Tensor) -> Tuple[Tensor, Tensor]: ...

```

This replaces the current pattern where every experiment script has a 100+ line `if/elif` chain selecting models.

Unified training loop One `Trainer` class handles: - Classification (cross-entropy + accuracy) or regression (MSE + MAE) - Warmup → cosine LR schedule - Best-validation checkpointing - Periodic + init + last checkpoint saving - CSV + JSON result logging - Configurable via **Hydra** (hierarchical YAML, `--multirun` sweeps, structured experiment composition)

This replaces the 10 duplicated training scripts in the Python codebase and 27 in the Julia codebase.

Batched ablation via torch.vmap The 15+ SRNN ablation variants (srnn, srnn-no-adapt, srnn-sfa-only, srnn-std-only, srnn-E-only, etc.) differ only in which dynamic parameters are enabled. With `torch.vmap`, we can: 1. Create a batch of B models with different configs 2. Stack their parameters into batched tensors (B, n, n), (B, n), etc. 3. Run a single `vmap(forward_fn)(batched_params, inputs)` call 4. Get B outputs simultaneously on one GPU using tensor cores

This yields $\sim 10\text{--}50\times$ **throughput** vs. sequential CPU training, and enables sweeping across model sizes (64, 128, 256, 512 neurons) simultaneously.

Dale’s law + sparsity as config options The SRNN’s `dales=True`, `per_neuron=True`, and `indegree` parameters become YAML config entries:

```
# configs/model/srnn.yaml
name: srnn
hidden_size: 128
n_E_ratio: 0.5
n_a_E: 3
n_a_I: 3
n_b_E: 1
n_b_I: 1
dales: true
per_neuron: false
indegree: null # null = full connectivity
solver: semi_implicit
ode_unfolds: 6
h: 0.0025 # dt = 1/400
```

7. Model Scaling Strategy

Current → Target sizes

DECIDED: Log-spaced model sizes from 4 to 1024. 256 is the expected sweet spot for batched ablation on a single L4 GPU.

Parameter	Legacy (v1/v2)	v3 target
Hidden size	32	round(logspace(4, 1024)) 4, 8, 16, 32, 64, 128, 256, 512, 1024
Batch size	16	64–256 (GPU)
Precision	float32	bfloat16 (tensor cores)
Training	CPU sequential	L4 GPU + batched ablations
Sequence length	16–32	96–16,000 (dataset-dependent)
Seeds per condition	5	5–10 (cheaper per seed on GPU)

Memory estimates for L4 (24 GB)

For a single SRNN model in bfloat16: - W: $512 \times 512 \times 2$ bytes = **0.5 MB** - State: $\text{batch}=128 \times \text{state_dim}=512 \times (1+3+3+1+1) \approx 5K \times 2$ bytes $\times 128$ = **1.3 MB** - Full model + optimizer: **~50 MB** for 512-neuron model

For batched ablation with B=15 variants: - 15×50 MB = **~750 MB** — easily fits in 24 GB with room for activations

Conclusion: L4’s 24 GB is more than sufficient for even aggressive batching of 512-neuron models across all ablation variants.

8. Cloud Infrastructure: GPU VMs on GCP

8.1 VM Configuration

DECIDED: g2-standard-16 (1× L4, 16 vCPUs, 64 GB RAM).

Replace the current CPU-only n2-standard-2 VMs with:

```
# GPU VM: g2-standard-16 (1x L4, 16 vCPUs, 64 GB RAM)
gcloud compute instances create ${VM_NAME} \
  --machine-type=g2-standard-16 \
  --zone=us-central1-a \
  --accelerator=type=nvidia-l4,count=1 \
  --image-family=pytorch-latest-gpu \
  --image-project=deeplearning-platform-release \
  --boot-disk-size=100GB \
  --scopes=compute-rw,storage-full \
  --metadata=... \
  --maintenance-policy=TERMINATE
```

8.2 Cost Comparison

Config	Per VM/hr	Full run (200 epochs × 7 datasets × 5 seeds × 6 models)
Current: n2- standard-2 (CPU)	\$0.10	~\$500 (long wall time per job)
Proposed: g2- standard-16 (L4)	~\$1.00	~\$50–150 (batched, much faster per job)

The GPU cost-per-compute-hour is 10× higher, but **wall-time savings** of 10–50× (from GPU acceleration + batched ablations) make the total cost **5–10× cheaper**. The extra CPU/RAM headroom of g2-standard-16 helps with data loading and preprocessing for larger datasets (Electricity 321-ch, AMASS, electrophysiology).

8.3 Cloud Workflow

The existing cloud infrastructure ([CloudNotes.md](#)) is well-designed and can be adapted: 1. **Keep:** GCS results bucket, self-deleting VMs, master-launcher pattern, monitor.sh 2. **Change:** Machine type → g2-standard-16, image → pytorch-latest-gpu, startup.sh installs from pyproject.toml 3. **Add:** Batched ablation mode — one VM runs all ablation variants for a dataset/seed combo

9. Mamba / SSM Baseline Integration

DECIDED: Both Mamba-2 and Mamba-3 from the start.

9.1 Models to Compare Against

Model	Source	Notes
Mamba-2	<code>pip install mamba-ssm</code>	Well-tested on L4 (SM89). Selective state space model.
Mamba-3	Source install from <code>state-spaces/mamba</code>	SISO/MIMO prefill works on L4; decode may need <code>mamba3-minimal</code> fallback. See MAMBA3_tech_stack.md . Install both optimized and <code>mamba3-minimal</code> fallback from day 1.
Gated Delta Network	Official PyTorch impl	Mamba-2 backbone with adaptive memory gating. Strong on retrieval and long-context tasks.
S4/S5	<code>state-spaces/s4</code>	Structured state space baseline. Linear recurrence.

Model	Source	Notes
LSTM	<code>torch.nn.LSTM</code>	Universal baseline.

9.2 Integration Strategy

Wrap each external model in our `BaseRNNCell` interface:

```
# src/models/mamba_wrapper.py
class MambaWrapper(nn.Module):
    """Wraps mamba_ssm.Mamba as a sequence model matching our interface."""
    def __init__(self, config):
        self.mamba = Mamba(d_model=config.hidden_size, d_state=config.d_state, ...)
    def forward(self, x): # x: (batch, seq_len, d_input)
        return self.mamba(self.input_proj(x))
```

This lets us run **all models** (LTC, SRNN, CT-RNN, LSTM, Mamba-2/3, Gated Delta, S4) through the same training loop, same datasets, same metrics.

10. Migration Path

Phase 1: Core framework (Weeks 1–2)

- ☐ Create `v3-liquid-networks/` repo with Hydra project structure
- ☐ Port `SRNNCell` from TF1 to PyTorch (translate from `srnn_model.py`)
- ☐ Port `LTCCell` (from `ltc_model.py` or use `ncps`)
- ☐ Port `CTRNNCell`, `NODECell`, `CTGRUCell` (from `ctrnn_model.py`)
- ☐ Implement unified `Trainer` class with Hydra config
- ☐ Implement HAR and Cheetah datasets (Tier 0 smoke tests)
- ☐ Validate against legacy results (same model size, same data → same accuracy)

Phase 2: GPU acceleration + new datasets (Weeks 3–4)

- ☐ Add BF16 mixed-precision training
- ☐ Implement `vmap`-based batched ablation runner
- ☐ Add ETTh1 and Electricity dataset loaders
- ☐ Add AMASS dataset loader (SMPL pose parameters)
- ☐ Integrate Mamba-2 and Mamba-3 wrappers (both from the start)
- ☐ Test on L4 GPU VM (g2-standard-16)
- ☐ Benchmark throughput: legacy CPU vs L4 GPU

Phase 3: Full benchmarks + SSM comparisons (Weeks 5–6)

- ☐ Add sMNIST, Speech Commands, and sCIFAR-10 datasets
- ☐ Add electrophysiology HDF5 dataset loader (512 Hz, up to 200 channels)
- ☐ Run comparison tables: SRNN vs LTC vs Mamba-2/3 vs LSTM on all Tier 1 datasets
- ☐ Adapt cloud infrastructure for GPU VMs
- ☐ Gated Delta Network integration

Phase 4: Scaling + cross-subject experiments (Weeks 7+)

- ☐ Scale SRNN across log-spaced sizes (4→1024 neurons)
- ☐ Run batched ablation sweeps (15 variants × 5 seeds × 9 sizes) at 256-neuron sweet spot
- ☐ Multi-subject / cross-subject electrophysiology experiments
- ☐ Human3.6M dataset (when academic license obtained)
- ☐ Full paper-ready results tables

11. Decisions Made (2026-03-24)

All major architecture decisions have been finalized:

#	Decision	Choice
1	Repo structure	Fresh repo: v3-liquid-networks/ (clean break from legacy TF1 and Julia codebases)
2	Primary framework	PyTorch — broadest ecosystem, Mamba/GDN native, ncps for LTC/CfC, torch.vmap/bmm for batched ablation
3	Config system	Hydra — hierarchical YAML, --multirun sweeps, structured experiment composition
4	Smoke test datasets	HAR (classification) + Cheetah (vector autoregression). sMNIST moves to Tier 1 (final benchmark, not smoke test).
5	Benchmark datasets	ETTh1 , Electricity , AMASS (account created), Speech Commands v2 , sCIFAR-10 , sMNIST . Human3.6M later (academic license pending).
6	Electrophysiology data	Yes — available . HDF5 v7.3 Matlab format, 512 Hz, up to 200 channels, 24 hours per subject, multiple subjects. Vector autoregression task. Start with one subject, expand to multi-subject.
7	SSM baselines	Both Mamba-2 and Mamba-3 from the start. Install optimized kernels + mamba3-minimal fallback.
8	Julia	Set aside for now . May revisit for adjoint method experiments later. Existing Julia/Matlab repos remain available.
9	Model sizes	round(logspace(4, 1024)) 4, 8, 16, 32, 64, 128, 256, 512, 1024 neurons. 256 is the expected sweet spot for batched ablation on one L4.
10	Cloud VM	g2-standard-16 (1× L4, 16 vCPUs, 64 GB RAM). Extra CPU/RAM for large dataset loading.

12. References

Existing project docs

- [Julia_ecosystem_analysis_opus.md](#) — Julia SciML for continuous-time RNNs
- [Julia_ecosystem_chatgpt.md](#) — Deep comparison of Julia options
- [MAMBA3_tech_stack.md](#) — Mamba-3 consumer GPU compatibility
- [mamba3-gpu-compatibility-t4-l4-a100.md](#) — T4/L4/A100 comparison
- [CloudNotes.md](#) — Current cloud infrastructure

Framework and model references

- [mamba-ssm](#) — Mamba-1/2/3 official repository
- [mamba3-minimal](#) — Pure PyTorch Mamba-3 fallback
- [ncps](#) — Neural Circuit Policies (LTC/CfC in PyTorch/Keras)
- [torchdiffeq](#) — PyTorch Neural ODE solvers
- [DiffraX](#) — JAX ODE/SDE/CDE solvers

Dataset references

- [ETT datasets](#) — Electricity Transformer Temperature

- [AMASS](#) — Archive of Motion Capture as Surface Shapes (account created; 40+ hrs, 344 subjects, SMPL body model params)
- [Human3.6M](#) — Motion capture benchmark (academic license pending)
- [Speech Commands v2](#) — 35-class audio classification
- [Long Range Arena](#) — Sequence modeling benchmark suite (archived Feb 2025)

Papers

- Hasani et al., “Liquid Time-constant Networks” (AAAI 2021)
- Hasani et al., “Closed-form Continuous-time Neural Networks” (Nature Machine Intelligence 2022)
- Gu & Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces” (2023)
- Yang et al., “Gated Delta Networks: Improving Mamba2 with Delta Rule” (2025)
- Arora et al., “Mamba-3: Sequentially Matching SSM, In-Context, and Decode Efficiency” (ICLR 2026)